GUIDE D’EXÉCUTION ET DE VALIDATION

Concurrent Ticket Booking Engine — Java 21

Hamza Mebrouk · Andame Amine · El Kasmi Mohammed · Ahmed Rida Bouraya

Un guide pas à pas, basé sur les captures réelles de notre groupe. Les commandes sont adaptées au terminal macOS et au projet com.concert.

Code couleur : titres principaux rouges, sous-titres verts, explications noires.

Avant de commencer

Ouvrir le terminal dans le dossier qui contient pom.xml :

```
cd "/Users/mac/Downloads/ticket-booking-java21 4"
```

Vérifier la présence de pom.xml avec : ls pom.xml

Toutes les commandes java ci-dessous supposent une compilation Maven préalable.

Parcours des tests

Ordre	Test	Commande / action
01	JUnit	mvn clean test
02	Race condition	Main race
03	Corrections	Main fixes
04	Pipeline	Main pipeline
05	Platform / Virtual	Main platform / Main virtual
06	Deadlock	Main deadlock
07	Menu interactif	Main interactive
08	Réservation et refus	Options 2 et 3
09	Paiement et expiration	Options 5 et 6
10	Statistiques et arrêt	Options 4, 1 et 7

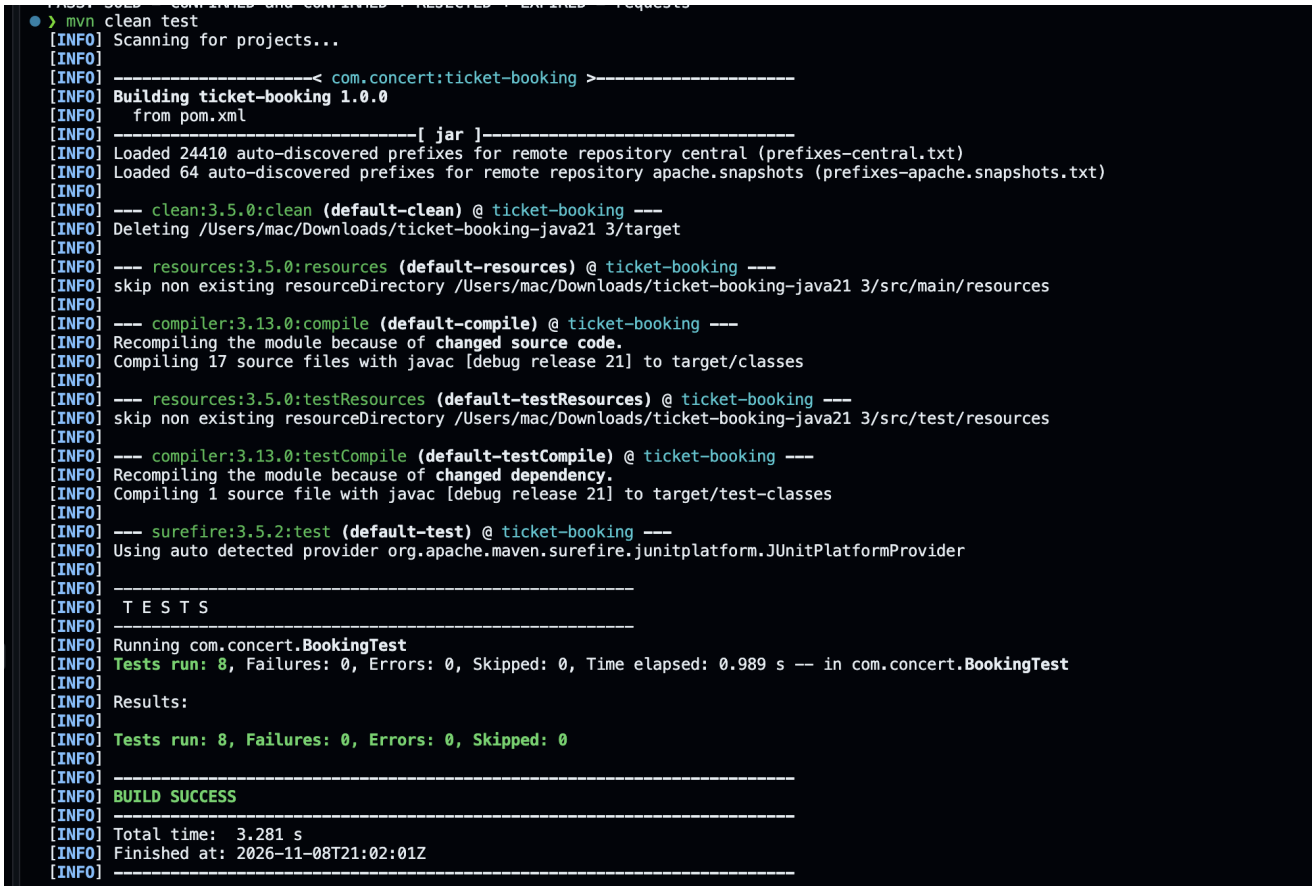
La suite présente la commande, la capture et une interprétation. Les nombres sont ceux de nos exécutions, pas ceux d’un autre groupe.

01 — Compilation et tests JUnit

COMMANDE À EXÉCUTER

```
mvn clean test
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

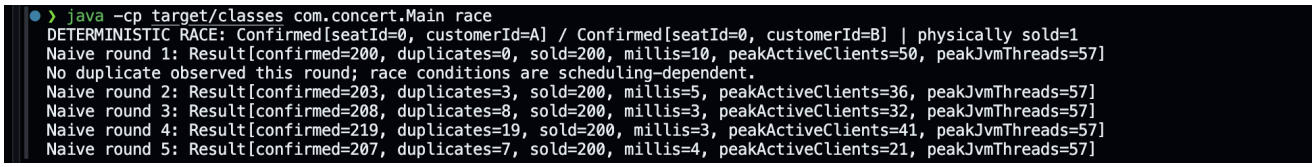
8 tests exécutés, aucune erreur, aucun échec et BUILD SUCCESS. Cela valide les cas couverts par les tests automatisés.

02 — Démonstration de la race condition

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main race
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

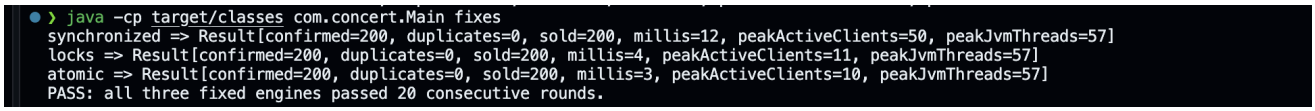
Le moteur naïf autorise parfois plusieurs confirmations pour une même place. Les cinq tours montrent 0, 3, 8, 19 et 7 doublons.

03 — Comparaison des trois corrections

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main fixes
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

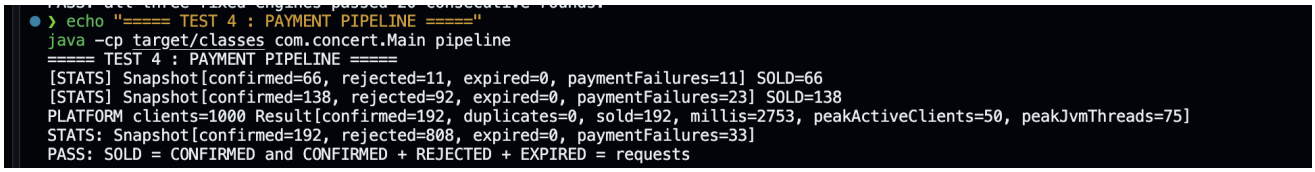
Les stratégies synchronized, ReentrantLock et Atomic CAS donnent 200 ventes et zéro doublon. Les 20 répétitions passent.

04 — Pipeline de paiement

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main pipeline
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

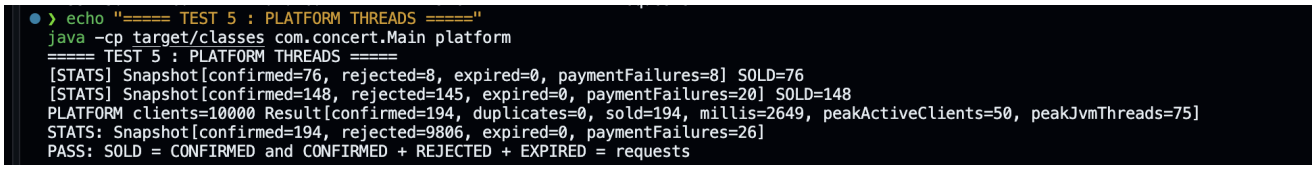
Sur 1 000 demandes : 192 confirmations et 808 refus. Le nombre de places vendues correspond aux confirmations, sans doublon.

05 — Stress test : Platform Threads

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main platform
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

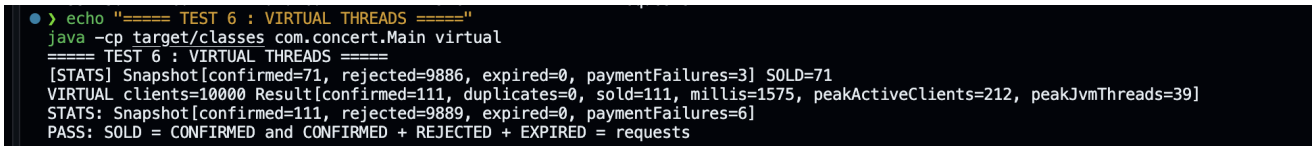
Sur 10 000 clients : 194 confirmations, 9 806 refus et 0 doublon ; durée de 2 649 ms pour cette exécution.

06 — Stress test : Virtual Threads

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main virtual
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

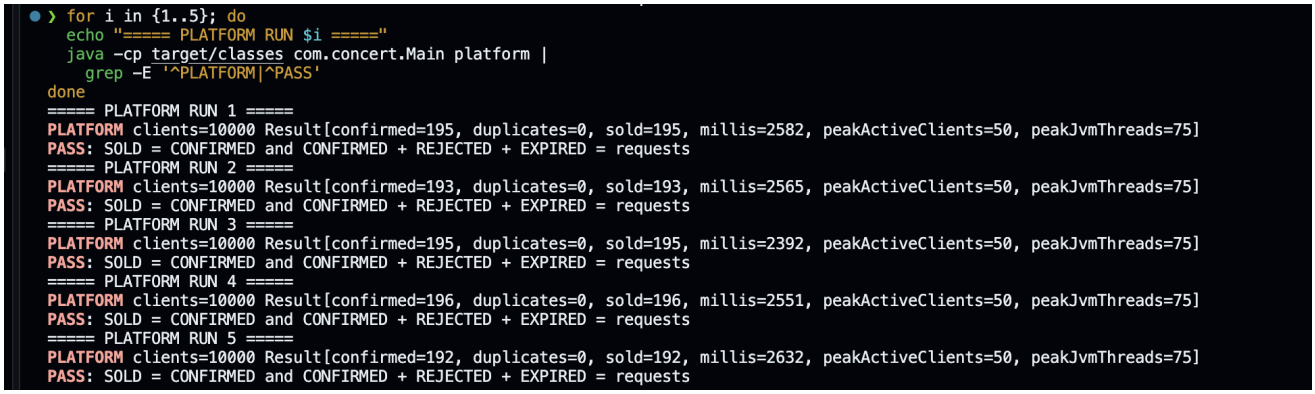
Sur 10 000 clients : 111 confirmations, 9 889 refus et 0 doublon ; durée de 1 575 ms pour cette exécution.

07 — Cinq répétitions : Platform Threads

COMMANDE À EXÉCUTER

```
for i in {1..5}; do java -cp target/classes com.concert.Main platform | grep -E "^PLATFORM|^PASS"; done
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

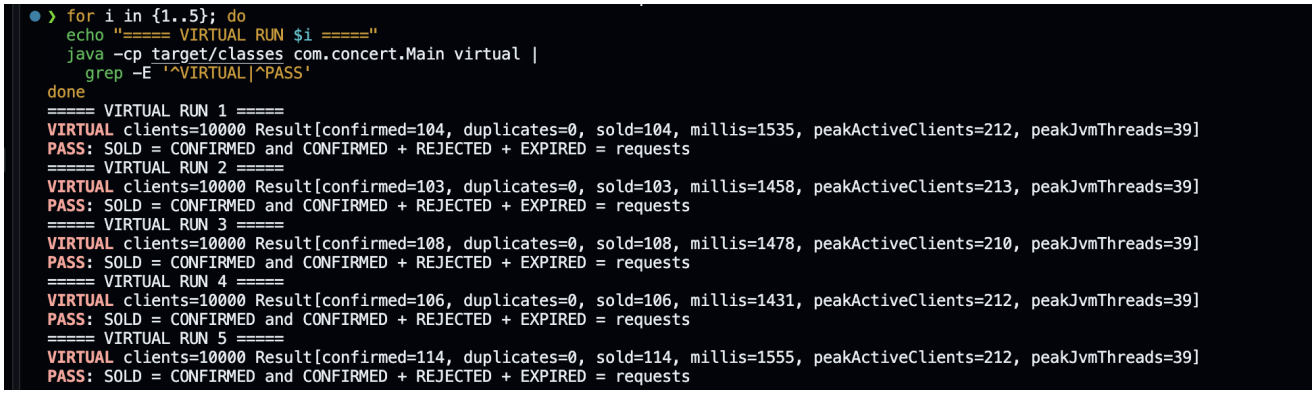
Durées : 2582, 2565, 2392, 2551 et 2632 ms. Moyenne : 2544,4 ms.

08 — Cinq répétitions : Virtual Threads

COMMANDE À EXÉCUTER

```
for i in {1..5}; do java -cp target/classes com.concert.Main virtual | grep -E
"^VIRTUAL|^PASS"; done
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

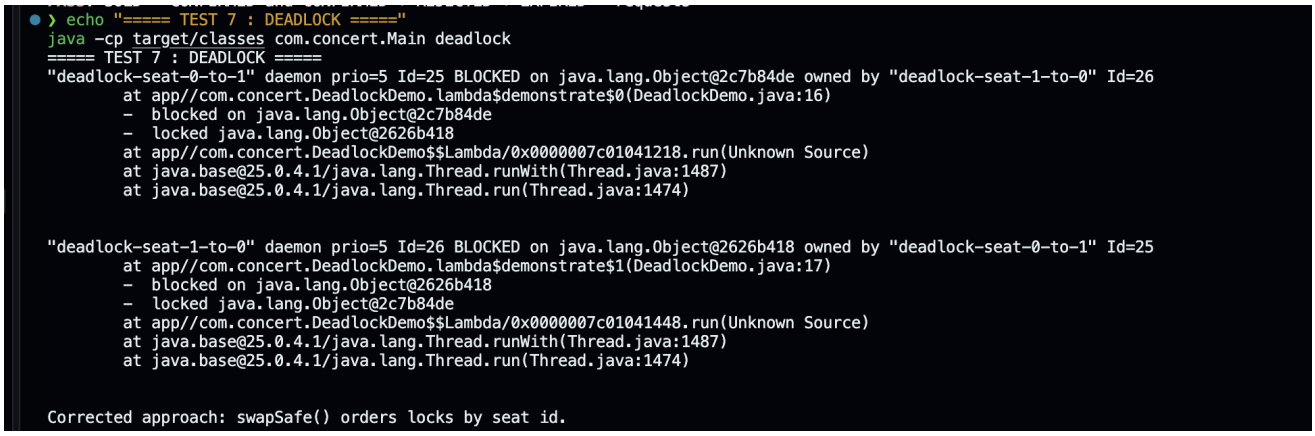
Durées : 1535, 1458, 1478, 1431 et 1555 ms. Moyenne : 1491,4 ms. Comparer les temps, sans oublier que les ventes confirmées diffèrent.

09 — Deadlock : deux verrous opposés

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main deadlock
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

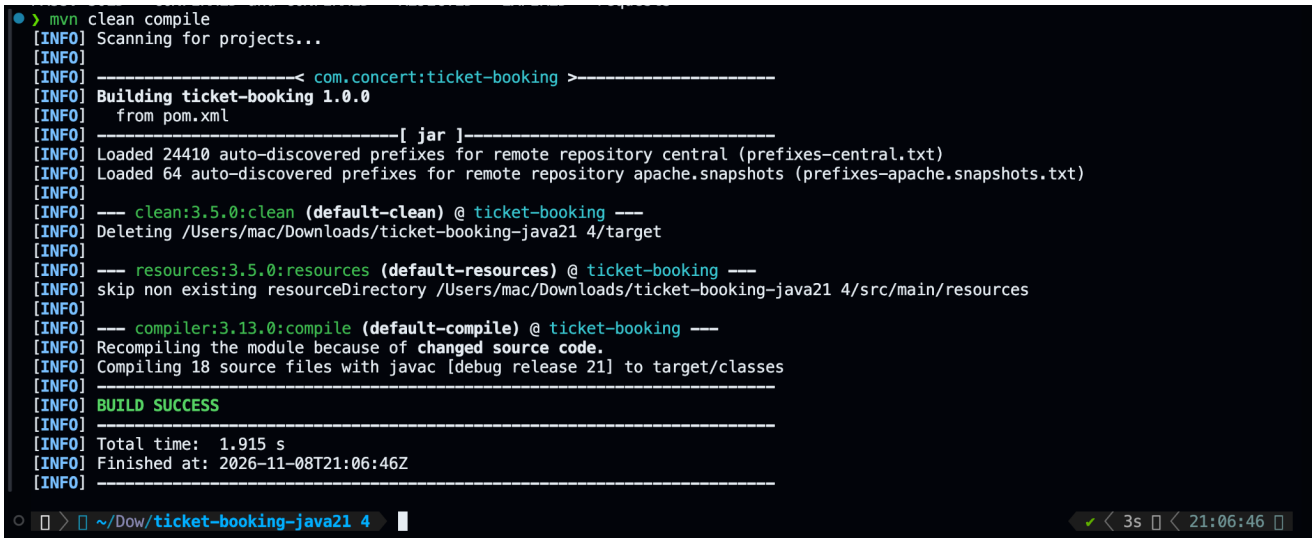
Deux threads BLOCKED attendent chacun un verrou détenu par l’autre. Le message swapSafe() présente le principe de correction par ordre des verrous.

10 — Compilation de la version interactive

COMMANDE À EXÉCUTER

```
mvn clean compile
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

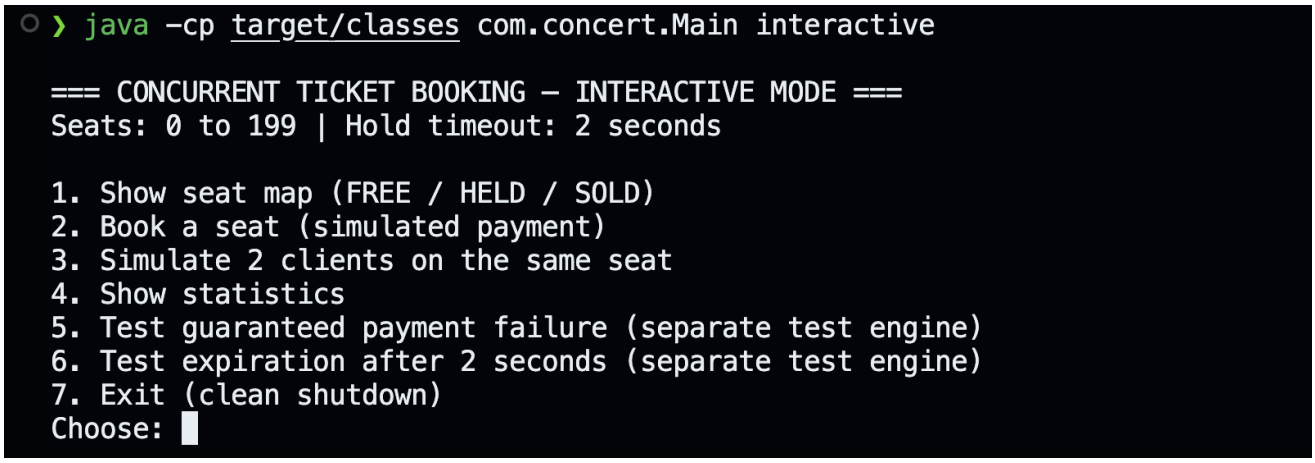
Les 18 fichiers Java sont compilés sans erreur. BUILD SUCCESS confirme que les classes sont générées dans target/classes.

11 — Lancement du menu interactif

COMMANDE À EXÉCUTER

```
java -cp target/classes com.concert.Main interactive
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

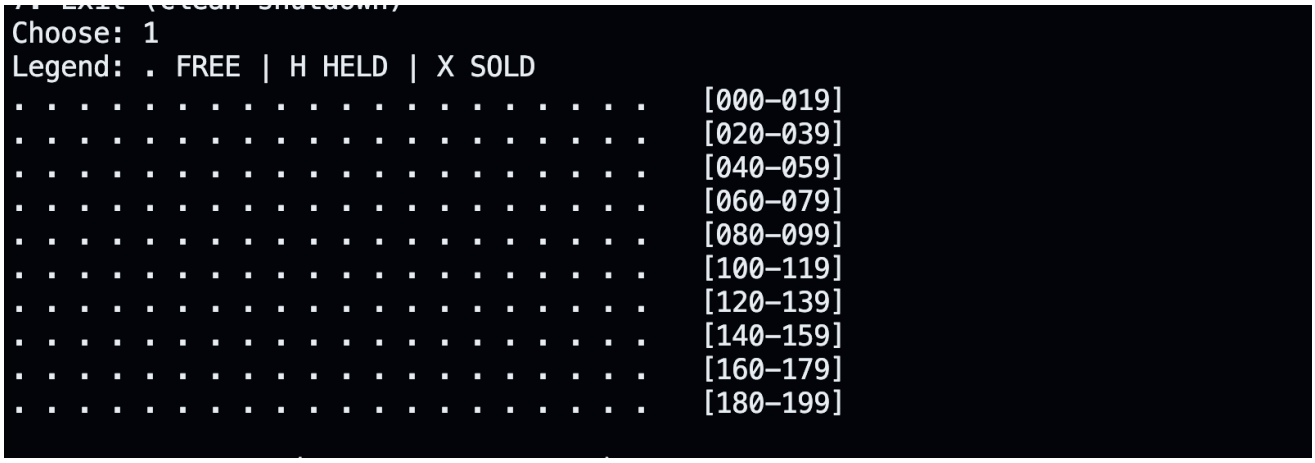
Le menu propose sept choix pour manipuler les places et déclencher des scénarios contrôlés.

12 — Carte initiale des 200 places

COMMANDE À EXÉCUTER

```
Menu interactif : option 1
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

Toutes les places de 0 à 199 sont FREE au démarrage. La légende distingue FREE, HELD et SOLD.

13 — Réservation réussie

COMMANDE À EXÉCUTER

`Menu : option 2, place 23, client amine`

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 2
Seat ID (0–199): 23
Customer name: amine
Booking result: Confirmed[seatId=23, customerId=amine]
Seat 23 now: SOLD
```

EXPLICATION DU RÉSULTAT

Confirmed[seatId=23, customerId=amine] : la place 23 passe à SOLD.

14 — Paiement échoué

COMMANDE À EXÉCUTER

```
Menu : option 5
```

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 5
Payment result: Rejected[reason=payment failed] | Seat state: FREE
PASS: failed payment released seat
```

EXPLICATION DU RÉSULTAT

Le paiement est refusé et la place revient à FREE. Le scénario affiche PASS.

15 — Expiration et paiement tardif

COMMANDE À EXÉCUTER

```
Menu : option 6
```

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 6
After 2s timeout: Expired[seatId=0] | Seat state: FREE
PASS: expired hold released seat
After late payment: FREE
PASS: late payment cannot sell expired seat
```

EXPLICATION DU RÉSULTAT

Après 2 secondes : Expired et place FREE. Le paiement tardif ne peut plus vendre cette place.

16 — Refus d’une double réservation

COMMANDE À EXÉCUTER

`Menu : option 2, place 23, client aymen`

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 2
Seat ID (0-199): 23
Customer name: aymen
Booking result: Rejected[reason=taken]
Seat 23 now: SOLD
```

EXPLICATION DU RÉSULTAT

Rejected[reason=taken] : la place 23 reste SOLD. Une deuxième vente est empêchée.

17 — Deux clients sur une place vendue

COMMANDE À EXÉCUTER

`Menu : option 3, place 23`

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 3
Seat ID (0-199): 23
Client A: Rejected[reason=taken]
Client B: Rejected[reason=taken]
Seat 23: SOLD
PASS: no double sale
```

EXPLICATION DU RÉSULTAT

Les deux demandes sont refusées. Ce scénario confirme le refus sur une place SOLD, mais ne teste pas une course sur une place FREE.

18 — Statistiques du moteur

COMMANDE À EXÉCUTER

```
Menu : option 4
```

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 4
Stats: Snapshot[confirmed=1, rejected=3, expired=0, paymentFailures=0]
Sold: 1 / 200
Payments queued: 0
```

EXPLICATION DU RÉSULTAT

Le moteur principal affiche 1 confirmation, 3 refus, 0 expiration, 1 place vendue et 0 paiement en attente.

19 — Carte après réservation

COMMANDE À EXÉCUTER

```
Menu : option 1
```

CAPTURE DU TERMINAL

```
1. Show seat map (FREE / HELD / SOLD)
2. Book a seat (simulated payment)
3. Simulate 2 clients on the same seat
4. Show statistics
5. Test guaranteed payment failure (separate test engine)
6. Test expiration after 2 seconds (separate test engine)
7. Exit (clean shutdown)
Choose: 1
Legend: . FREE | H HELD | X SOLD
. . . . . . . . . . . . . . . . . . . .   [000-019]
. . . X . . . . . . . . . . . . . . . .   [020-039]
. . . . . . . . . . . . . . . . . . . .   [040-059]
. . . . . . . . . . . . . . . . . . . .   [060-079]
. . . . . . . . . . . . . . . . . . . .   [080-099]
. . . . . . . . . . . . . . . . . . . .   [100-119]
. . . . . . . . . . . . . . . . . . . .   [120-139]
. . . . . . . . . . . . . . . . . . . .   [140-159]
. . . . . . . . . . . . . . . . . . . .   [160-179]
. . . . . . . . . . . . . . . . . . . .   [180-199]
```

EXPLICATION DU RÉSULTAT

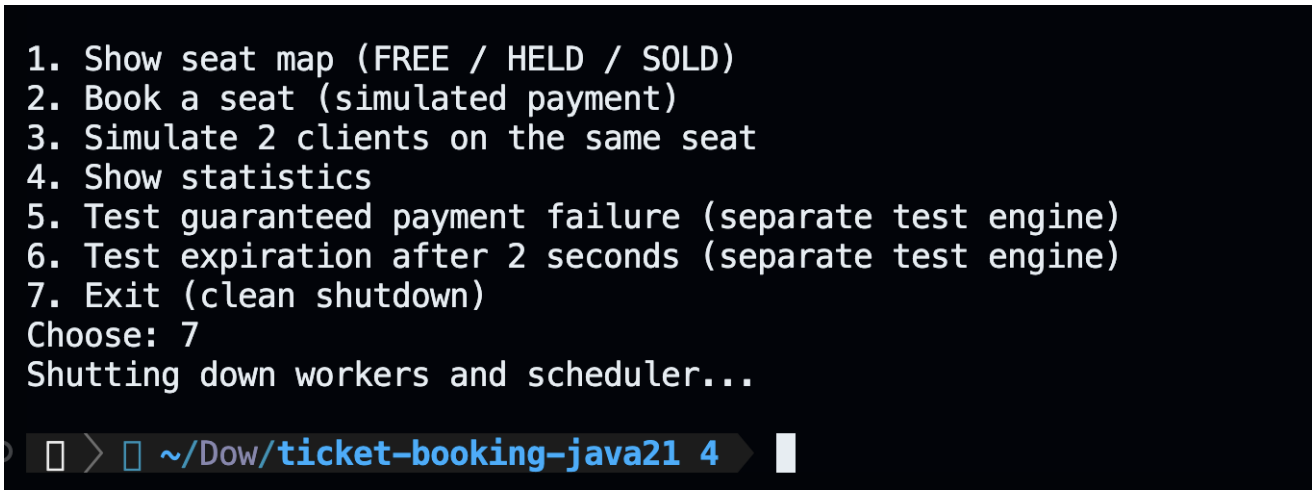
Un X apparaît à la place 23, correspondant à la place SOLD ; les autres places restent libres.

20 — Sortie et arrêt du programme

COMMANDE À EXÉCUTER

```
Menu : option 7
```

CAPTURE DU TERMINAL



EXPLICATION DU RÉSULTAT

Le programme affiche la fermeture des workers et du scheduler puis rend la main au terminal. Cela prouve la sortie, pas un drain de queue sous charge.

Bilan des résultats

Le projet démontre le problème de concurrence, trois corrections sans doublon, la gestion des paiements et expirations, les stress tests, un interblocage et une interface de validation.

Performances mesurées sur cinq essais : Platform Threads = 2 544,4 ms ; Virtual Threads = 1 491,4 ms. Dans ce scénario, les threads virtuels réduisent le temps moyen d’environ 41,4 %. Les charges ont le même nombre de clients mais pas le même nombre de confirmations.

Limites à expliquer pendant la soutenance

Le test interactif à deux clients utilise une place déjà vendue. Le deadlock corrigé, le vidage d’une queue saturée, le pinning et un thread dump externe ne sont pas démontrés par ces captures. Les 8 tests JUnit ne doivent pas être présentés comme une couverture exhaustive.

Conclusion : les captures permettent de présenter les fonctions effectivement exécutées et vérifiées, avec des résultats reproductibles à partir des commandes du guide.